

SYSTEM TECHNICAL DOCUMENTATION

Project Performance Improvement Toolkit

Architecture, components, and deployment

Technical source: oli_v6_deploy.py

Updated: May 11, 2026

1. Scope and Source of Truth

This technical documentation describes only the main application file, oli_v6_deploy.py. The files oli_v6_deploy_core.py and oli_v6_deploy_recommendations.py are separate variants and are not used as factual sources for this English handoff.

The main application is a monolithic Streamlit app. It contains helper functions, document extraction, AI-based evaluation, recommendation classification, visualization, and export logic in one Python file.

2. High-Level Architecture

Layer	Implementation in Code	Responsibility
Interface	Streamlit with st.tabs, st.session_state, st.cache_data, and institutional ILO CSS	Document upload, criteria selection, visualizations, and downloads.
Document processing	python-docx, docx2python, BeautifulSoup, and hierarchical heading extraction	Converts DOCX/TXT into traceable text by section, paragraph, table, and metadata.
Generative AI	OpenAI SDK through OpenAI(api_key=os.getenv('OPENAI_API_KEY'))	Evaluations, synthesis, document chat, answer criticism, and executive summaries.
Vector search	text-embedding-3-large, numpy, torch, and FAISS IndexFlatIP	Loads a FAISS recommendation index at startup and uses FAISS for large-document RAG in Tab 4; Tabs 5 and 6 classify with numpy cosine similarity, not FAISS.
Data	Excel files, pipe-separated CSV, and .pt tensors in the app directory or paths defined in st.secrets	There is no SQL database; files are the operational data store.
Export	xlsxwriter, BytesIO, and zipfile	Creates XLSX and ZIP outputs with results, evidence, extracted structure, and reports.

3. Active Functional Components

Tab	Visible Name	Main Function
1	Valoracion Preliminar de Calidad de Proyectos	Evaluates project design documents against Appraisal

		Checklist_2025 es-419.xlsx, applies A-E analysis and the A/B rule for transversal matters, and generates critique and synthesis by question, subsection, and section.
2	Diagnostico de Atributos Especificos	Evaluates selected sections with rubrics from Rubricas_6ago2025.xlsx for participation during evaluation, gender, and modern just transition.
3	Diagnostico de Sostenibilidad del Proyecto	Evaluates sustainability with Evaluacion de sostenibilidad del proyecto_rubric_9feb26.xlsx and visualizes scores by dimension and criterion.
4	Preguntale a tus Documentos	Chats with one or more DOCX/TXT files, using full context for small documents and FAISS RAG for large documents.
5	Clasificacion de Recomendaciones	Classifies recommendations from a Spanish interface against Frame_Recommendations_English.xlsx; the required recommendation-file field is Recommendation description.
6	Recommendation Classification	English recommendation-classification interface using the same classification logic as Tab 5.

4. Dependencies

requirements.txt pins or declares these main dependencies:

```
streamlit==1.27.0
pandas==2.0.3
numpy==1.24.3
faiss-cpu==1.7.4
openai>=1.0.0
matplotlib==3.7.2
seaborn==0.12.2
torch==2.0.1
plotly==5.15.0
xlsxwriter==3.1.2
python-dotenv==1.0.0
openpyxl
rich==13.9.0
python-docx==0.8.11
docx2python
tiktoken>=0.5.0
```

5. AI Models and Context Limits

- Main model for evaluation, critique, synthesis, and chat: gpt-5-mini.
- Embeddings model: text-embedding-3-large.
- Deep recommendation analysis displays a selector between gpt-4o-mini and gpt-4o, but the current code does not pass that selection into run_row_analysis; analyze_recommendation_plan_pair uses gpt-4o-mini by default.
- Operational document limit: truncate_to_token_limit cuts text to roughly 110,000 tokens to leave room for prompt and response.
- In document chat, total texts up to 100,000 characters use full context and are then cut to 110,000 tokens; larger texts are split into 2,000-character chunks with 300-character overlap and searched with FAISS RAG.

5.1 Two-Part Question Rule in Tab 1

Tab 1 detects compound questions through parse_two_part_question, including a statement followed by a question, explicit separators such as 'especificamente', two question marks, and run-together cases. When two parts are detected, Part 2 is the evaluation focus and Part 1 is only contextual framing.

- The first analysis reasoning must begin with 'Se identificaron 2 partes en esta pregunta.'
- Evidence is tagged [DEDICATED] when it contributes specific elements for the evaluated subject and [FRAMING] when it is only context, a group list, or general inclusion language.
- The Tab 1 critic re-evaluates with the full document and the code applies a mechanical gate to the verdict.
- If the model does not keep focus on Part 2, the row can remain with Status = Partial even if analysis completed.

5.2 Transversal-Matter Rule in Tab 1

Tab 1 keeps the general A-E framework for questions with a specific subject, but bypasses it completely when the question matches a configured transversal matter.

Recognized Transversal Matter	Detection Criteria in Code
Gender	Aliases such as genero, igualdad de genero, enfoque de genero, mujeres, and ninas.
Non-discrimination	Aliases such as no discriminacion, antidiscriminacion, discriminacion, igualdad de oportunidades, and trato igualitario.
Disability	Aliases such as discapacidad, personas con discapacidad, PCD, accesibilidad, and inclusion de personas con discapacidad.
Social dialogue and tripartism	Aliases such as dialogo social, tripartismo, tripartita, sindicatos, organizaciones de trabajadores, and organizaciones de empleadores.
Environmental sustainability	Aliases such as sostenibilidad ambiental, medio ambiente, ambiental, cambio climatico, accion climatica, and economia verde.

Transversal Criterion	Operational Definition
A	Operational presence of the matter in an objective, output, or activity.
B	Budget, resources, or budget line for a

	corresponding activity.
--	-------------------------

The automatic grade for these matters is: Yes when A and B are present; Partial when only A or only B is present; No when neither A nor B is present. Indicators and targets do not count for the transversal rule. The critic can mechanically overwrite a verdict that is inconsistent with A/B.

5.3 Precise Tab 4 Behavior

- DOCX is extracted with python-docx by reading paragraphs; TXT is read as UTF-8 plain text with errors='ignore'. This flow does not use the enriched hierarchical extractor, so tables, heading metadata, pages, and section-level citations are not reliable in Tab 4.
- For large documents, chunk embeddings are stored in st.session_state; for each question the app rebuilds a FAISS IndexFlatIP, normalizes vectors, and retrieves up to 15 vector chunks.
- RAG context also adds up to 5 chunks by exact keyword match, deduplicated against vector-retrieved chunks.
- The model call includes only the last 5 chat messages.
- File-change detection uses the list of file names, not content hashes; uploading a different file with the same name can leave previous state until the session is restarted or the file is renamed.
- The small-document path requires answers only from the document; the RAG path permits inference with contextual clues, so conclusions must be verified against the source file.

6. Initialization and Cache

- load_data() loads df_complete_all_full.xlsx and df_split_actions.xlsx and prepares normalized columns.
- load_extended_data() adds analyzed_recommendations_plans_v5.csv when available and stores analyzed_df in st.session_state.
- load_embeddings() loads emb_Recomm_rec_cl_4.pt and Recommendation_RAG_Metadata.pt and builds an in-memory FAISS index. Although the visible recommendation-search interface is commented out, this load remains an active dependency because it runs at startup.
- Load functions use st.cache_data; clearing Streamlit cache forces files to reload.
- The main block initializes data and embeddings twice; final behavior is equivalent, but a future cleanup should consolidate it.
- Tabs 5 and 6 use embeddings_cache.pkl for classification embeddings and analysis_cache.pkl for deep recommendation analysis.
- Tab 4 stores documents, chunks, and embeddings only in st.session_state during the session.

7. Deployment

Standard local execution:

```
streamlit run oli_v6_deploy.py
```

In production, when STREAMLIT_ENV=production, the code reads file paths from st.secrets. The OpenAI key is read from OPENAI_API_KEY.

Secret or Variable	Use in Code
OPENAI_API_KEY	Authenticates the OpenAI client.
df_path	Production path to df_complete_all_full.xlsx.
df_raw_path	Production path to df_split_actions.xlsx.
embeddings_path	Production path to emb_Recomm_rec_cl_4.pt.

structured_embeddings_path	Production path to Recommendation_RAG_Metadata.pt.
analyzed_recommendations_path	Optional production path to analyzed_recommendations_plans_v5.csv.
lessons_embeddings_path	Path to emb_LL_IL_cl_4.pt for a lessons helper that is not connected to an active tab.
structured_lessons_path	Path to lessons_metadata.pt for a lessons helper that is not connected to an active tab.

8. Operational Technical Risks

- The app does not implement its own authentication, authorization, audit trail, tenant isolation, uploaded-file scanning, or automatic secret redaction; it relies on the hosting environment for access control.
- Uploaded documents and extracted text may be sent to OpenAI for analysis and must be treated as potentially sensitive.
- The data store is file-based; replacing an input file changes app behavior without schema migrations.
- Local paths are relative to the directory from which Streamlit is run.
- Generated XLSX/ZIP outputs can contain verbatim citations from source documents and require the same confidentiality treatment.
- Local pickle caches are neither encrypted nor integrity-checked.